SL StudyLoop

Study smarter, not harder.

An AI-assisted flashcard app with spaced repetition — end-term project proposal & build plan.

COURSE	BATCH	STACK
Building Web Applications with React	2029	React · Firebase · Tailwind · Vercel

OVERVIEW

Executive Summary

THE PROBLEM

Students cram before exams and forget 70% within a week. No tool turns their own notes into a spaced-repetition system.

THE PRODUCT

StudyLoop — create decks, review flashcards daily, and get smart schedules via the SM-2 algorithm.

THE USERS

College & high-school students preparing for exams, certifications, and long-term courses.

THE DELIVERABLE

Production-grade React app with Firebase Auth + Firestore, deployed live on Vercel.

Why students forget what they study

70%

forgotten within a week without review.

— Ebbinghaus forgetting curve

Scattered material

Notes in 3 notebooks, slides in Drive, PDFs in 6 tabs.

Passive re-reading

Feels productive but builds no long-term memory.

Cramming

Last-minute push → short-term recall, forgotten after exam.

Existing tools (Anki)

Powerful but a 1998 UI and no AI card generation.

StudyLoop — three connected features

1

Decks & Cards

Organise your study

- Create decks by subject or topic
- Add Q/A cards manually
- Optional: paste notes → AI generates cards
- Edit, delete, search, filter

2

Study Session

Spaced repetition (SM-2)

- Daily 'Due today' queue
- Flip cards, rate recall 1–4
- Algorithm schedules next review
- Keyboard-driven for speed

3

Dashboard & Stats

See your progress

- Current + longest streak
- Mastery % per deck (chart)
- 14-day activity trend
- 8-week heatmap grid

Who we're building for

R

Riya, 19

2nd-year engineering student

Juggles	6 subjects · 3 exams/sem
Studies on	laptop + phone (commute)
Needs	structure, not more content
Avoids	setup-heavy tools

Why this matters

Active recall + spaced repetition is the most evidence-backed learning technique in cognitive science.

Yet 9 out of 10 students still rely on re-reading and highlighting — methods shown to be the weakest.

StudyLoop packages proven science into a simple daily routine:

open the app → clear today's queue → close.

Chosen tools & why

FRONTEND

React 18

Core library — functional components + hooks

Vite

Fast dev server, HMR, modern ESM build

React Router v6

Client-side routing + protected routes

BACKEND

Firebase Auth

Email/password + Google OAuth out of the box

Cloud Firestore

Real-time NoSQL DB with onSnapshot listeners

Security Rules

Per-document access control (user-scoped)

UI & UX

Tailwind CSS

Utility-first, dark mode, design tokens

Framer Motion

Spring physics for card flip animation

Recharts

Responsive SVG charts for stats dashboard

TOOLING

Lucide Icons

Consistent SVG icon set (no emoji)

Vercel

Zero-config deploy, preview URLs per branch

Git + GitHub

Conventional commits, clean history

Focused-Minimal visual system

COLOUR TOKENS

Primary	Success	Streak	Danger	Text	Border
#4F46E5	#10B981	#F59E0B	#F43F5E	#0F172A	#E2E8F0

TYPOGRAPHY

Plus Jakarta Sans

Headings · Bold · Tracks tight

Inter — crisp, neutral, readable

Body · 15–16pt · 1.5 line-height

DESIGN PRINCIPLES

✓ **Light + dark mode**

Both themes from day one — study happens at night.

✓ **Motion with meaning**

150–250ms ease-out; spring only on card flip.

✓ **Keyboard-first**

Every action reachable without a mouse.

✓ **One CTA per screen**

Primary indigo button; secondary outlined.

Firestore collections & relationships

users/

uid

- email
- displayName
- streak
- lastStudiedAt
- theme

decks/

deckId

- userId
- title
- description
- color
- createdAt

cards/

cardId

- deckId
- userId
- question
- answer
- ease
- interval
- dueAt
- reps
- lapses

reviews/

reviewId

- userId
- cardId
- rating
- reviewedAt

SECURITY

Firestore Security Rules enforce request.auth.uid == resource.data.userId on every read and write — no user can ever see another user's decks, cards, or reviews.

React concepts → where each lives

CONCEPT	WHERE USED
Functional components	Every file in /components and /pages
useState	Card form, auth forms, study session queue
useEffect	Data fetch, auth listener, theme sync
Props & composition	Deck → DeckCard, Session → FlashCard
Conditional rendering	Auth gate, empty states, loading, errors
Lists & keys	Decks grid, cards list, review history
Lifting state up	Study session owns rating, cards emit events
Controlled components	All forms (deck, card, auth, settings)
Context API	AuthContext · ThemeContext · ToastContext
React Router	8 routes, protected + dynamic /decks/:id
useMemo	Filtered decks, mastery % calc, due queue
useCallback	Handlers passed to memoised children
useRef	Focus card form on open, session timer
React.lazy + Suspense	/study, /stats, /settings lazy-loaded
Custom hooks	useAuth, useDecks, useCards, useStudySession

Ten parts, ~2.5 hours focused build

1	2	3	4	5
Scaffold	**Firebase wiring**	**Auth**	**Layout + Theme**	**Decks CRUD**
10m	5m	15m	15m	15m
6	7	8	9	10
Cards CRUD	**Study session**	**Dashboard + Stats**	**Polish + A11y**	**Docs + Deploy**
10m	20m	15m	20m	15m

★ *Part 7 (Study Session) is the centrepiece and will be demoed live.*

Scaffold & design tokens

OBJECTIVE

Stand up the Vite + React project with Tailwind, routing, folder structure, and design tokens. Landing header renders with correct fonts and colours.

DELIVERABLE

Project boots with npm run dev and shows a styled landing header. Folder structure matches the rubric's /components, /pages, /hooks, /context, /services requirement.

YOU CAN DEMO

Boot screen with StudyLoop brand and fonts.

KEY FILES

- package.json, vite.config.js (path alias @/)
- tailwind.config.js (indigo, emerald, amber, slate tokens)
- index.css (base layer, typography, focus ring)
- src/{components, pages, hooks, context, services, lib, layouts}
- App.jsx, routes.jsx, main.jsx

REACT CONCEPTS

Functional components | Composition

Firebase wiring

OBJECTIVE

Connect Firebase Auth + Firestore. Write security rules, seed script, and environment variable template. All services ready for feature work.

DELIVERABLE

Importing { db, auth } from services/firebase works throughout the app. Rules file is ready to paste into Firebase Console, and the seed script creates demo data on a single command.

YOU CAN DEMO

Firestore console shows empty collections with rules active.

KEY FILES

- .env.example, .env.local
- services/firebase.js (initialiseApp, auth, db)
- firestore.rules (owner-only per document)
- firestore.indexes.json (dueAt composite)
- scripts/seed.js (2 demo decks, 20 cards)

REACT CONCEPTS

Service layer | Config

Authentication

OBJECTIVE

Implement Email/Password + Google OAuth with session persistence. Every app route gated by a ProtectedRoute wrapper. First-time signup creates user doc.

DELIVERABLE

User can sign up, log out, log back in, refresh the page, and remain signed in. Google OAuth works. Unauthenticated users are redirected to /login.

YOU CAN DEMO

Complete signup → redirect → logout → re-login loop.

KEY FILES

- pages/Login.jsx, pages/Signup.jsx
- context/AuthContext.jsx (user, loading, signIn, signOut)
- components/ProtectedRoute.jsx, PublicOnlyRoute.jsx
- services/auth.js (all Firebase auth calls)
- hooks/useAuth.js

REACT CONCEPTS

- Context API
- useEffect
- Controlled forms
- Conditional rendering
- useState

App layout & theming

OBJECTIVE

Build the persistent app shell: sidebar (desktop) / bottom nav (mobile), top bar with streak + theme toggle, global toast system, and light/dark mode.

DELIVERABLE

Fully responsive shell that works on 375px → 1440px. Theme toggle persists across reloads. Active route is highlighted in the nav.

YOU CAN DEMO

Toggle theme, resize window, navigate between placeholder pages.

KEY FILES

- layouts/AppLayout.jsx (sidebar + top bar)
- context/ThemeContext.jsx (light | dark, localStorage + Firestore)
- components/{Spinner, EmptyState, ErrorBoundary, Toast}
- pages/NotFound.jsx

REACT CONCEPTS

- Context API
- useEffect
- Conditional rendering
- Lists & keys

Decks CRUD

OBJECTIVE

Users create, read, update, and delete decks in real time. Grid view with search, sort, and colour-coded cards. Deck detail page.

DELIVERABLE

Create, rename, re-colour, and delete decks. Changes propagate across tabs in real time via onSnapshot. Search + sort respond instantly.

YOU CAN DEMO

Create 3 decks → rename one → delete another → confirm real-time update in a second tab.

KEY FILES

- pages/Decks.jsx, pages/DeckDetail.jsx
- components/DeckCard.jsx, DeckForm.jsx, ConfirmDialog.jsx
- hooks/useDecks.js (onSnapshot listener)
- services/decks.js (createDeck, updateDeck, deleteDeck, listDecks)

REACT CONCEPTS

useState	useEffect	useMemo
Lists & keys	Controlled forms	Props

Cards CRUD

OBJECTIVE

Inside a deck, add / edit / delete flashcards. Inline editing, keyboard shortcuts, and a bulk-paste helper that splits multi-line text into cards.

DELIVERABLE

Add 10+ cards to a deck in under a minute. Cmd/Ctrl+Enter saves; bulk paste accepts 'Q: ...\nA: ...' blocks.

YOU CAN DEMO

Bulk paste 5 cards → edit one inline → delete one.

KEY FILES

- components/CardRow.jsx, CardForm.jsx, BulkPaste.jsx
- hooks/useCards.js (deck-scoped onSnapshot)
- services/cards.js (CRUD + batch write for bulk)
- lib/parsePaste.js (heuristic splitter)

REACT CONCEPTS

- useRef
- useState
- Controlled forms
- Lists & keys
- useCallback

Study session · SM-2 algorithm

OBJECTIVE

The centrepiece feature. Pull due cards, flip with a spring animation, rate recall, and let the SuperMemo-2 algorithm schedule the next review.

DELIVERABLE

Full study flow: due queue → flip → rate 1/2/3/4 → next card → completion summary with streak update. All keyboard-navigable.

YOU CAN DEMO

Complete a full session with keyboard only.

KEY FILES

- pages/Study.jsx (/study/:deckId)
- components/FlashCard.jsx (Framer Motion flip)
- hooks/useStudySession.js (queue state machine)
- lib/sm2.js (pure algorithm, unit-testable)
- services/reviews.js (write review + update card)

REACT CONCEPTS

useState	useEffect	useCallback
useRef	Lifting state	Custom hooks

Dashboard & stats

OBJECTIVE

Home screen with streak, due-today CTA, and recent decks. Stats page with mastery bar chart, 14-day activity line, and 8-week heatmap grid.

DELIVERABLE

Visually rich dashboard with Recharts. Streak and mastery update live as the user studies. Empty states for new users.

YOU CAN DEMO

Dashboard after a study session — streak +1, charts update.

KEY FILES

- pages/Home.jsx, pages/Stats.jsx
- components/StreakBadge, DueTodayCard, MasteryChart, ActivityHeatmap
- hooks/useStats.js (aggregates reviews)
- lib/stats.js (streak + mastery calculations)

REACT CONCEPTS

- useMemo
- useEffect
- Lists & keys
- Conditional rendering

Polish · AI · Settings · A11y

OBJECTIVE

Add the optional AI card generator, build the Settings page, lazy-load heavy routes, and complete a full accessibility sweep.

DELIVERABLE

AI generation works when an API key is set, degrades gracefully otherwise. Settings supports export + account deletion. All routes keyboard-accessible; contrast ≥ 4.5:1; prefers-reduced-motion respected.

YOU CAN DEMO

Paste notes → generate cards → review with screen-reader friendly nav.

KEY FILES

- services/ai.js (Gemini call, graceful fallback)
- pages/Settings.jsx (profile, theme, export, delete)
- components/GenerateFromNotes.jsx
- App.jsx — React.lazy + Suspense for /study, /stats, /settings

REACT CONCEPTS

React.lazy | Suspense | useMemo | useCallback | Custom hooks

Docs & deploy

OBJECTIVE

Write the README, clean up commits, deploy to Vercel, and prepare the 3–5 minute demo video.

DELIVERABLE

Live URL on Vercel. Clean git history with conventional commits. README explains the problem, concepts used, and setup in under 2 minutes.

YOU CAN DEMO

Live URL loaded on phone + desktop.

KEY FILES

- README.md (problem, features, stack, setup, deploy)
- docs/SCREENSHOTS/*, docs/DEMO_SCRIPT.md
- .github/workflows/ci.yml (optional lint + build)
- vercel.json (env vars + SPA rewrite)

REACT CONCEPTS

Documentation | Build output

How each mark is earned

CRITERIA	MARKS	HOW WE EARN IT
Problem Statement & Idea	15	Original AI + SM-2 combo, clear student user, non-trivial
React Fundamentals	20	Every core + intermediate concept demonstrated (slide 9)
Advanced React Usage	15	useMemo, useCallback, useRef, lazy/Suspense, custom hooks
Backend Integration	15	Firebase Auth + Firestore + RLS + full CRUD across 3 collections
UI / UX	10	Design tokens, dark mode, responsive, keyboard nav, empty/loading/error
Code Quality	10	/components /pages /hooks /context /services · small files · clean commits
Functionality	10	3 core features work end-to-end, handled edge cases
Demo & Explanation	5	Scripted 4-minute walkthrough hitting problem → stack → code
TARGET	**100**	***All criteria covered by the 10-part plan***

Let's build it.

Ten parts. One focused build. A portfolio-ready React app.

→ **Create Firebase project**

Enable Email/Password, Google, Firestore

→ **Paste firebaseConfig**

Into .env.local before Part 2

→ **Say “go”**

I start Part 1 immediately